



Praktikum Pemrograman Mobile

Pertemuan 5:

## ***Networking & Architecture***

Disusun oleh :

**Azis Amirulbahar, S.P.d., M.T.I.**

**Program Studi Informatika**

**Fakultas Teknik**

**Universitas Jenderal Soedirman**

**2026**



# DAFTAR ISI

|                                                                                      |          |
|--------------------------------------------------------------------------------------|----------|
| <b>DAFTAR ISI</b> .....                                                              | <b>2</b> |
| <b>PENDAHULUAN</b> .....                                                             | <b>3</b> |
| A. Pengantar .....                                                                   | 3        |
| B. Tujuan Praktikum .....                                                            | 3        |
| C. Alat dan Bahan .....                                                              | 4        |
| D. Dasar Teori.....                                                                  | 4        |
| <b>TAHAPAN PRAKTIKUM</b> .....                                                       | <b>5</b> |
| A. Berkenalan dengan JSON.....                                                       | 5        |
| B. Penghapusan Dummy Data.....                                                       | 8        |
| C. Membuat Util .....                                                                | 10       |
| D. Menambahkan Permissions .....                                                     | 12       |
| E. Mendaftarkan Library Retrofit untuk Akses ke API .....                            | 12       |
| F. Membuat ApiService (Retrofit) .....                                               | 16       |
| G. Membuat ViewModel .....                                                           | 18       |
| H. Modifikasi DetailProductScreen.kt .....                                           | 21       |
| I. Modifikasi Daftar Produk .....                                                    | 25       |
| J. Memuat gambar yang bersumber dari internet .....                                  | 28       |
| K. Navigasi di Homeactivity untuk menangani klik dari daftar produk ke detail produk | 31       |

# PENDAHULUAN

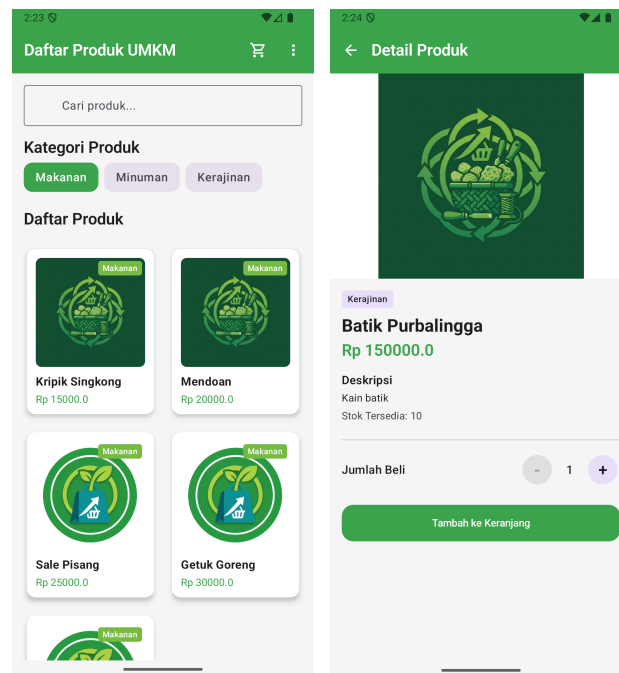
## A. Pengantar

Pada pertemuan ke-5 ini, kita akan mempelajari konsep lanjutan dari *Jetpack Compose* yaitu tentang **Networking & Architecture**. Pada pertemuan – pertemuan sebelumnya kita menggunakan daftar data yang dibuat secara *dummy* atau tiruan. Namun, dalam aplikasi yang sebenarnya, data yang diperoleh dan diolah oleh aplikasi diambil dari *web services* melalui API (*Application Programming Interface*) dengan umumnya menggunakan media JSON (*JavaScript Object Notation*) sehingga pengembang/*developer* memerlukan koneksi ke jaringan *internet* (*networking*). Aplikasi yang melakukan pengolahan data memerlukan baris kode yang lebih panjang, sehingga organisasi/penyusunan struktur kode/file dalam *source code* perlu dilakukan agar mudah dikembangkan, dibaca oleh tim, maupun saat pemeliharaan.

## B. Tujuan Praktikum

1. Mahasiswa memahami cara mengambil data JSON dari server menggunakan pustaka Retrofit.
2. Mahasiswa memahami dan mengimplementasikan pola arsitektur MVVM (*Model-View-ViewModel*).
3. Mahasiswa memahami konsep StateFlow untuk merepresentasikan keadaan UI (UI State) seperti *Loading*, *Success*, dan *Error*.

Hasil Akhir dari praktikum adalah sebagai berikut:



Gambar 1. Hasil Akhir Praktikum

## C. Alat dan Bahan

1. Perangkat keras (PC/Laptop) dengan spesifikasi minimum sesuai standar *Android Studio*.
2. *Android Studio* (versi terbaru yang sudah mendukung *Jetpack Compose* secara bawaan).
3. Emulator Android (AVD) atau perangkat fisik *Android* yang terhubung melalui *debugging* (USB/Wi-Fi).

## D. Dasar Teori

1. **Retrofit & Gson:** Dalam pengembangan *android* yang memerlukan akses data melalui *webservices/API*, diperlukan *library* untuk mengambil dan memproses data yang dihasilkan. Salah satu *library* yang sering digunakan oleh para *developer/pengembang android* adalah Retrofit. Retrofit merupakan *library* untuk berinteraksi dengan API RESTful dan mengunduh data dari server. Retrofit secara otomatis mengkonversi respons data (umumnya berupa JSON) menjadi objek Kotlin yang mudah digunakan dengan **converter Gson**.

2. **MVVM** (*Model-View-ViewModel*) adalah pola arsitektur perangkat lunak yang memisahkan kode program menjadi tiga lapisan utama agar aplikasi lebih terstruktur, mudah dirawat, dan mudah diuji. Lapisan pertama adalah **Model**, yang bertugas mengelola data murni dan logika bisnis—seperti mengambil data dari server atau basis data lokal. Lapisan kedua adalah **View**, yaitu antarmuka visual yang dilihat dan digunakan oleh pengguna yang berfungsi menampilkan data pengguna tanpa memproses logika apa pun. Lapisan ketiga adalah **ViewModel**, yang berfungsi sebagai jembatan penghubung antara Model dan View untuk melakukan pemrosesan data/logika.

## TAHAPAN PRAKTIKUM

Pada bagian ini, peserta diminta untuk mengikuti tahapan – tahapan sehingga menghasilkan aplikasi sederhana seperti pada gambar 1 di halaman sebelumnya.

### A. Berkenalan dengan JSON

Pada praktikum ini, kita akan menggunakan data yang bersumber dari API (*Application Programming Interface*) dengan media JSON (*JavaScript Object Notation*) di mana data tersebut menggunakan web services sebagai perantaranya. **API (*Application Programming Interface*)** adalah jembatan komunikasi yang bertindak seperti pelayan restoran untuk menghubungkan aplikasi mobile (*frontend*) dengan server (*backend*), di mana aplikasi mengirimkan permintaan (*request*) melalui alamat tujuan khusus (*endpoint*) dan menerima kembali respon data (*response*) tanpa perlu mengakses basis data secara langsung. Sementara itu, **JSON (*JavaScript Object Notation*)** adalah format teks terstandar dan universal yang digunakan oleh API untuk “membungkus” data tersebut agar ringan dan mudah dipahami oleh berbagai bahasa pemrograman, menggunakan aturan sederhana berupa kurung kurawal { } untuk merepresentasikan satu objek data dan kurung siku [ ] untuk daftar kumpulan data (*array*) berbasis pasangan kunci dan nilai (*key-value pairs*).

Pada pertemuan ini, kita akan mengakses data melalui api berikut:

1. <https://pemmob-if.web.app/data/products.json>
2. <https://pemmob-if.web.app/data/categories.json>

Ketika aplikasi *mobile* ingin menghubungi server, aplikasi memerlukan alamat tujuan layaknya alamat rumah pengiriman. Alamat tersebut tersusun atas dua komponen utama: **Base URL** dan **Endpoint**. *Base URL* adalah domain utama dari server yang bersifat tetap, sedangkan *endpoint* adalah jalur spesifik di ujung URL yang mengarah ke sumber daya atau data tertentu yang ingin diambil. Pada praktikum ini, bagian <https://pemmob-if.web.app> berperan sebagai **Base URL**, yaitu alamat induk dari server yang dituju. Sementara itu, bagian setelahnya yaitu `/data/categories.json` dan `/data/products.json` adalah **Endpoint**, yaitu dua jalur terpisah yang masing-masing mengembalikan data spesifik berupa JSON kategori dan JSON produk. Di bawah ini adalah contoh data berformat JSON:

```
{ "total_produk": 2,

  "daftar_produk": [

    {

      "id": 1,

      "nama": "Kripik Singkong",

      "harga": 15000.0,

      "tersedia": true

    },

    { "id": 6,

      "nama": "Es Durian",

      "harga": 25000.0,

      "tersedia": true

    }

  ]

}
```

Berikut adalah tabel penjelasan struktur data JSON di halaman sebelumnya tersebut:

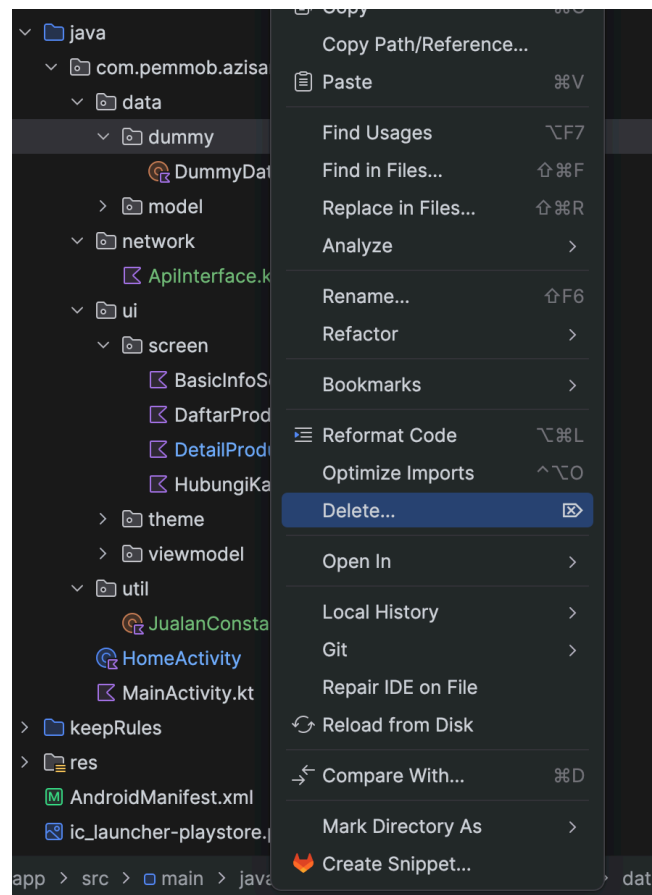
Tabel 1. Penjelasan Struktur JSON

| Komponen / Nama Key  | Nilai pada Data                   | Tipe Data Json          | Tipe Data (Kotlin)   | Keterangan                                                                               |
|----------------------|-----------------------------------|-------------------------|----------------------|------------------------------------------------------------------------------------------|
| { } (Root)           | Seluruh <i>payload</i> /isi data  | JSON Object             | Object / Map / Class | Objek utama pembungkus respon API berformat JSON.                                        |
| <b>total_produk</b>  | 2                                 | Integer                 | Int / int            | Metadata jumlah total barang yang tersedia di dalam daftar.                              |
| <b>daftar_produk</b> | [ ... ]                           | JSON Array              | List<Product>        | Kumpulan (list) objek produk untuk ditampilkan pada antarmuka/UI mobile.                 |
| <b>id</b>            | 1, 6                              | Integer                 | Int / int            | Identitas unik ( <i>Primary Key</i> ) setiap produk untuk referensi detail.              |
| <b>nama</b>          | "Kripik Singkong",<br>"Es Durian" | String                  | String               | Nama produk yang ditampilkan sebagai judul item pada UI.                                 |
| <b>harga</b>         | 15000.0,<br>25000.0               | Floating Point / Number | Double / double      | Nominal harga barang yang nantinya diformat ke mata uang (Rupiah).                       |
| <b>tersedia</b>      | true                              | Boolean                 | Boolean / bool       | Status ketersediaan stok; berguna untuk logika aktif/tidaknya tombol beli pada aplikasi. |

## B. Penghapusan Dummy Data

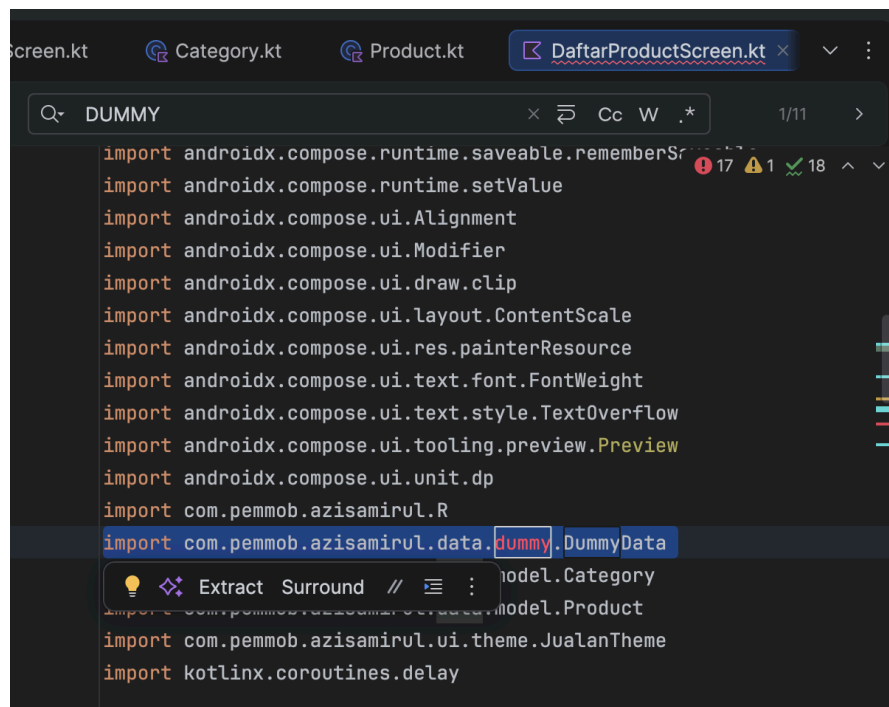
Pada tahapan ini, kita diminta untuk menghapus *package* maupun *Kotlin file* yang berisi *dummy data*. Pada praktikum sebelumnya, *dummy data* adalah sumber data yang di-hard coded (data statis).

1. Silakan klik kanan pada direktori *dummy* dan pilih *delete* seperti pada gambar berikut:



Gambar 2. Menghapus Package & File Data Dummy

3. Kemudian hapus baris – baris import yang menggunakan package dummy pada seluruh file Kotlin.

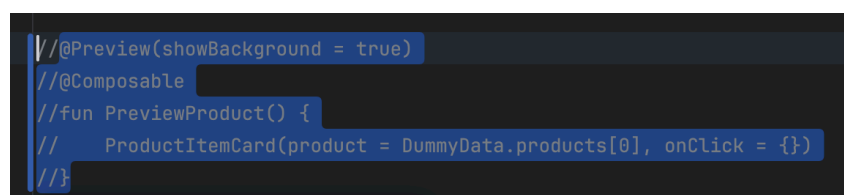


Gambar 3. Menghapus Import Package Dummy

**Mempraktikan tanda komentar pada kode:** Selain menghapus kode, praktik agar kode tidak dieksekusi oleh *compiler* adalah dengan memberikan tanda komentar. Pada bahasa pemrograman Kotlin, komentar ditandai dengan tanda `//` untuk kode 1 baris, atau dapat juga menggunakan tanda `/*` dan diakhiri `*/` apabila lebih dari 1 baris kode. **Silakan lakukan pemberian komentar pada kode – kode yang terdapat di beberapa file Kotlin yang menggunakan object `DummyData`, dengan cara blok baris kode yang akan diubah menjadi komentar lalu tekan tombol berikut:**

- **Windows/Linux:** Tekan `Ctrl + /*`.
- **Mac:** Tekan `Cmd + /*`.

Hasil kode yang telah diberi komentar adalah sebagai berikut:

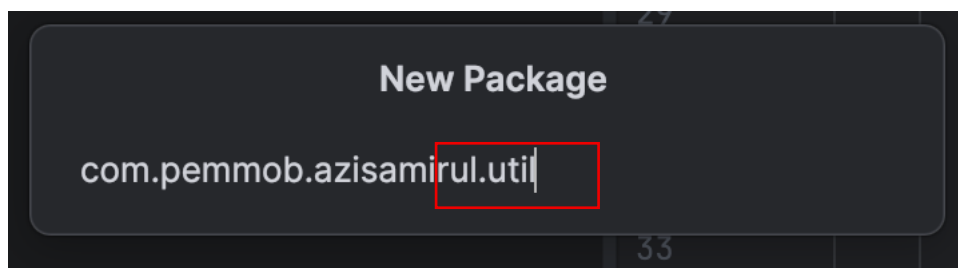


Gambar 4. Kode yang telah diberi komentar

### C. Membuat Util

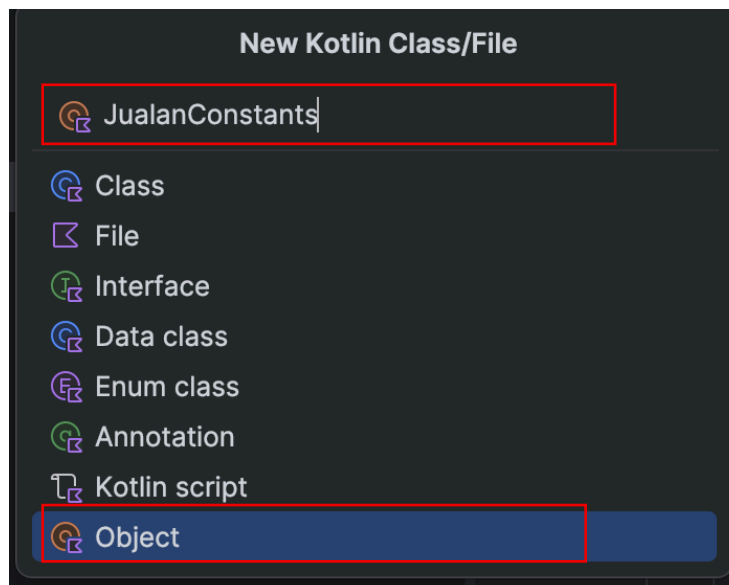
Dalam proyek *Android*, folder atau paket **util** (kependekan dari *utilities*) berfungsi sebagai wadah untuk menyimpan berkas atau kelas pembantu (*helper classes*) dan fungsi pendukung (*utility functions*) yang digunakan berulang kali di berbagai bagian aplikasi. Tujuan utamanya adalah menerapkan prinsip **DRY (Don't Repeat Yourself)**, sehingga kode yang sifatnya umum tidak perlu ditulis ulang di setiap **Activity**, **Fragment**, atau **ViewModel**.

1. Silakan buat *package* seperti pada gambar berikut:



Gambar 5. Membuat package util

2. Silakan buat *Kotlin file* di dalam *package util* dengan nama **JualanConstants** yang bertipe *Object* seperti pada gambar berikut:



Gambar 6. Membuat FileJualanConstants

### 3. Tuliskan kode berikut pada file *JualanConstants*:

```
object JualanConstants {  
    const val BASE_URL = "https://pemmob-if.web.app/"  
}
```

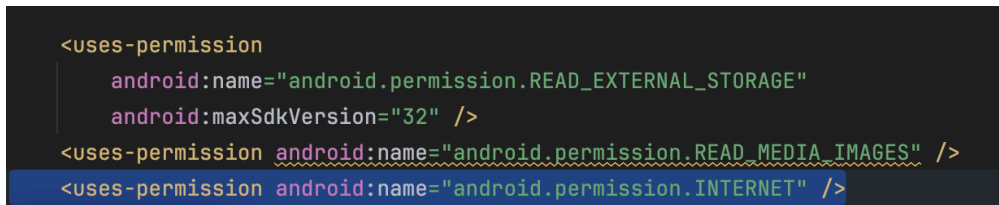
Gambar 7. Deklarasi Object *JualanConstants*

Potongan kode tersebut mendefinisikan sebuah objek tunggal (*singleton object*) di Kotlin bernama *JualanConstants* yang berfungsi sebagai wadah penyimpanan nilai-nilai konstan aplikasi secara terpusat. Di dalamnya, terdapat variabel *BASE\_URL* yang dideklarasikan menggunakan kata kunci *const val* serta menandakan bahwa nilai tersebut merupakan konstanta *compile-time* bertipe *String* yang bersifat tetap, tidak dapat diubah nilainya selama aplikasi berjalan, dan dapat diakses langsung secara global tanpa perlu membuat instansiasi objek baru (cukup dengan memanggil *JualanConstants.BASE\_URL*). URL <https://pemmob-if.web.app/> berperan sebagai alamat domain server yang nantinya digunakan oleh *library* sebagai titik awal sebelum digabungkan dengan berbagai *endpoint* spesifik (seperti */data/products.json* atau */data/categories.json*), sehingga jika sewaktu-waktu domain server terdapat perubahan, pengembang cukup memperbarui alamat URL pada satu baris kode tersebut tanpa harus mengubah seluruh deklarasi variabel.

## D. Menambahkan Permissions

Untuk dapat mengambil data dari pihak lain, kita perlu menambahkan permissions INTERNET . Secara bawaan, *platform android* melakukan isolasi terhadap jaringan luar.

1. Buka **AndroidManifest.xml** dan tambahkan permissions seperti pada gambar berikut:



```
<uses-permission
    android:name="android.permission.READ_EXTERNAL_STORAGE"
    android:maxSdkVersion="32" />
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />
<uses-permission android:name="android.permission.INTERNET" />
```

Gambar 8. Menambahkan Permissions Internet

Baris kode

```
<uses-permission android:name="android.permission.INTERNET" />
```

di dalam berkas **AndroidManifest.xml** berfungsi sebagai izin dari aplikasi kepada sistem operasi Android agar diizinkan mengakses jaringan internet. Secara bawaan, Android menerapkan sistem keamanan ketat yang mengisolasi setiap aplikasi demi melindungi privasi pengguna, sehingga aplikasi tidak dapat secara bebas berkomunikasi dengan *environment* luar.

## E. Mendaftarkan *Library Retrofit* untuk Akses ke API

Secara bawaan/*default*, *library/pustaka* tidak langsung terpasang pada proyek android yang kita buat. Sehingga, kita perlu melakukan konfigurasi untuk dapat menggunakan *library* yang kita butuhkan. *Library* dikembangkan oleh google maupun dapat pula dikembangkan oleh komunitas. Salah satu referensi pencarian *library* terdapat pada situs berikut: <https://android.libhunt.com/>

Silakan ikuti tahapan di bawah ini untuk melakukan penambahan library retrofit:

1. Buka file `gradle/libs.versions.toml`, dan tambahkan kode yang diblok warna biru seperti gambar di bawah ini yang terletak pada bagian `[versions]`:



```
[versions]
agp = "9.3.2"
coreKtx = "1.19.0"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
lifecycleRuntimeKtx = "2.11.0"
activityCompose = "1.13.0"
kotlin = "2.2.10"
composeBom = "2026.02.01"
navigationRuntimeKtx = "2.10.0"
navigationCompose = "2.10.0"
material3 = "1.4.0"
retrofit = "3.0.0"
```

Gambar 9. Mendeklarasikan versi Retrofit yang digunakan

`libs.versions.toml` adalah berkas konfigurasi bernama **Gradle Version Catalog**. Fungsinya adalah sebagai **pusat pengelolaan dependensi dan versi pustaka (*libraries*)** untuk seluruh modul di dalam proyek Android/Gradle. **TOML** adalah singkatan dari **Tom's Obvious, Minimal Language**. "Tom's": Merujuk pada Tom Preston-Werner yang merancang pertama kali pada tahun 2013.

Baris kode `retrofit = "3.0.0"` di bawah bagian `[versions]` pada berkas `libs.version.toml` berfungsi untuk mendefinisikan nomor versi dari pustaka (*library*) Retrofit yang digunakan di dalam proyek Android. File `libs.version.toml` merupakan fitur modern di Gradle bernama **Version Catalog**, yaitu tempat terpusat untuk mengelola seluruh daftar (*dependency*) agar nomor versi setiap *library* tidak berantakan dan tidak ditulis berulang secara manual di berbagai file `build.gradle`. Dengan menuliskan baris tersebut, sistem mencatat bahwa versi Retrofit yang dirujuk adalah "3.0.0", sehingga nantinya saat variabel versi ini dipanggil pada blok `[libraries]`, Gradle secara otomatis mengunduh Retrofit versi tersebut, dan jika suatu saat terdapat perubahan versi hanya bagian tersebut yang perlu dilakukan perubahan nomor versinya.

2. Masih dalam file `gradle/libs.versions.toml`, dan kode yang terdapat pada kotak berwarna merah berikut:

```
[libraries]
androidx-compose-material-icons-extended = { module = "androidx.compose.material:material-icons-extended" }
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref = "lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-espresso-material3 = { group = "androidx.compose.material3", name = "material3" }
androidx-navigation-runtime-ktx = { group = "androidx.navigation", name = "navigation-runtime-ktx", version.ref = "navigationRuntimeKtx" }
androidx-navigation-compose = { group = "androidx.navigation", name = "navigation-compose", version.ref = "navigationCompose" }
androidx-material3 = { group = "androidx.compose.material3", name = "material3", version.ref = "material3" }

retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
retrofit-converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "retrofit" }
```

Gambar 10. Menambahkan library retrofit

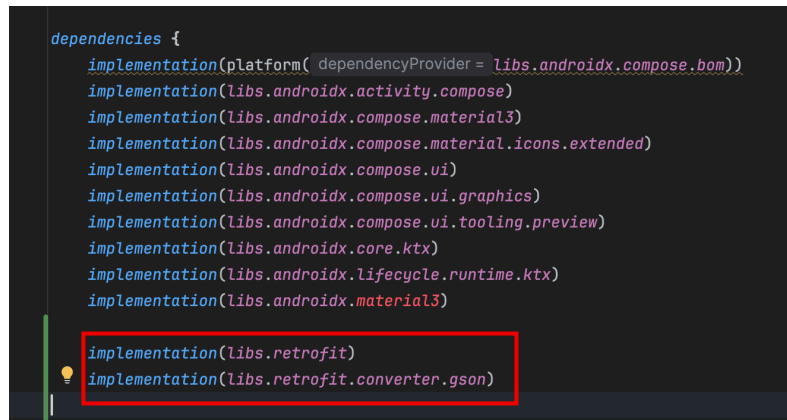
Atau dapat menyalin pada blok berikut:

```
retrofit = { module = "com.squareup.retrofit2:retrofit",
version.ref = "retrofit" }
retrofit-converter-gson = { module =
"com.squareup.retrofit2:converter-gson", version.ref =
"retrofit" }
```

Kedua baris kode tersebut berfungsi untuk mendaftarkan *library* jaringan beserta modul konverternya (*json* ke *data class Kotlin*) secara terpusat sebelum dipanggil di dalam proyek Android. Baris retrofit mendaftarkan library Retrofit (`com.squareup.retrofit2:retrofit`) yang bertugas mengelola koneksi HTTP, menangani pengiriman permintaan (*request*), dan menerima respons data dari server, dengan mengaitkan nomor versinya pada variabel `version.ref = "retrofit"` yang telah didefinisikan di blok `[versions]`.

Baris `retrofit-converter-gson` mendaftarkan modul Gson Converter (`com.squareup.retrofit2:converter-gson`) yang berfungsi menerjemahkan (*parsing*) format data secara otomatis dari teks mentah JSON menjadi objek model data Kotlin (*data class*) maupun sebaliknya.

3. Buka file **build.gradle.kts** yang terdapat di dalam folder **app**, kemudian tuliskan kode seperti pada gambar di bawah ini yaitu di blok **dependencies** yang terdapat di dalam kotak berwarna merah:



```
dependencies {  
    implementation(platform(dependencyProvider = libs.androidx.compose.bom))  
    implementation(libs.androidx.activity.compose)  
    implementation(libs.androidx.compose.material3)  
    implementation(libs.androidx.compose.material.icons.extended)  
    implementation(libs.androidx.compose.ui)  
    implementation(libs.androidx.compose.ui.graphics)  
    implementation(libs.androidx.compose.ui.tooling.preview)  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.lifecycle.runtime.ktx)  
    implementation(libs.androidx.material3)  
  
    implementation(libs.retrofit)  
    implementation(libs.retrofit.converter.gson)
```

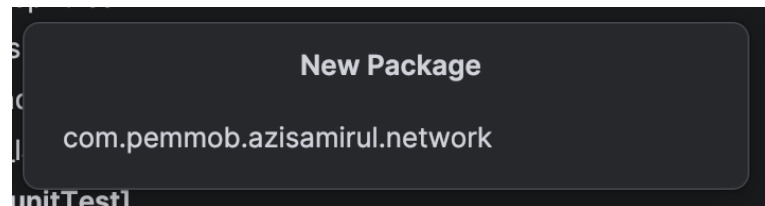
Gambar 11. Menambahkan implementation retrofit

Kedua baris kode tersebut wajib ditulis di dalam blok **dependencies** pada berkas **build.gradle.kts** karena berkas inilah yang menjadi acuan eksekusi utama bagi Gradle saat menyusun dan mengompilasi aplikasi Android. Jika berkas **libs.version.toml** diibaratkan sebagai buku daftar menu yang hanya mencatat nama dan versi *library*, maka **build.gradle.kts** adalah daftar belanja resmi yang memerintahkan Gradle untuk benar-benar mengunduh library **Retrofit** dan **Gson Converter** dari server pusat ke dalam project android kita. Kata kunci **implementation** bertindak sebagai instruksi tegas agar berkas biner (*library*) tersebut disertakan ke dalam *source code* aplikasi. Jika baris tersebut tidak ditulis, Android Studio tidak akan mengenali fungsi-fungsi di dalam library seperti `Retrofit.Builder()` atau `GsonConverterFactory`, yang dapat menjadi masalah saat kompilasi (*unresolved reference*) dan aplikasi tidak dapat dijalankan.

## F. Membuat ApiService (Retrofit)

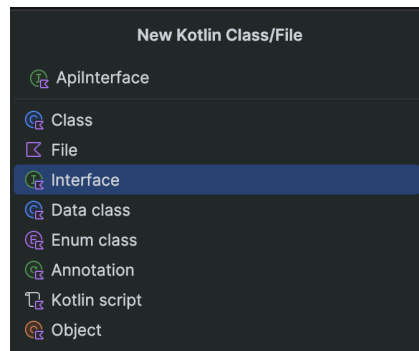
Pada bagian ini, kita akan mengimplementasikan kode yang merupakan function bawaan dari retrofit untuk digunakan dalam proses memanggil API dan melakukan penerjemahan JSON ke Data Class Kotlin.

### 1. Buatlah package bernama network



Gambar 12. Membuat Package Network

### 2. Di dalam package network, buat Kotlin file bernama ApiInterface dengan tipe Interface.



Gambar 13. Membuat Kotlin File bernama ApiInterface

### 3. Di dalam file `ApiInterface` tuliskan kode berikut:

```
2 Usages
interface ApiInterface
{
    1 Usage
    @GET("data/categories.json")
    suspend fun getCategories(): List<Category>

    1 Usage
    @GET("data/products.json")
    suspend fun getProducts(): List<Product>
}

3 Usages
object ApiClient {

    val instance: ApiInterface by lazy {
        Retrofit.Builder()
        .baseUrl(BASE_URL)
        .addConverterFactory(GsonConverterFactory.create())
        .build()
        .create(ApiInterface::class.java)
    }
}
```

Gambar 14. Implementasi kode `ApiInterface`

Potongan kode di atas merupakan implementasi arsitektur jaringan Retrofit di Kotlin yang memisahkan antara **k"ontrak" pemanggilan data (*interface*)** dan **konfigurasi klien HTTP**.

Interface dengan nama `ApiInterface` bertindak sebagai "kontrak kerja" yang mendefinisikan daftar fungsi pemanggilan data ke server beserta rutennya. Anotasi `@GET("data/categories.json")` dan `@GET("data/products.json")` memberi tahu Retrofit untuk melakukan *HTTP GET Request* ke masing-masing *endpoint*. Kata kunci `suspend` menandakan bahwa fungsi `getCategories()` dan `getProducts()` dirancang untuk berjalan di dalam Kotlin Coroutines, sehingga proses pengunduhan data dilakukan secara asinkron (*background thread*) tanpa memblokir atau membekukan tampilan antarmuka (*UI thread*).

Tipe kembalian/*return* yang bernama `List<Category>` dan `List<Product>` menegaskan bahwa data respons JSON berbentuk *array* nantinya akan otomatis diubah langsung menjadi daftar objek model Kotlin yang siap pakai.

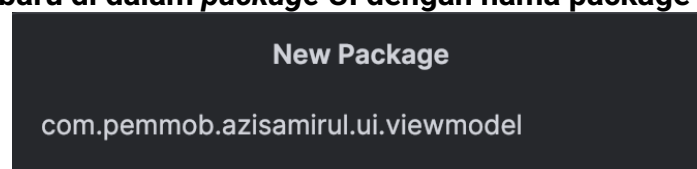
Pada bagian kedua, object `ApiClient` bertindak sebagai pusat inisialisasi tunggal (*singleton*) agar aplikasi tidak boros memori akibat membuat objek jaringan berulang kali. Properti `val instance: ApiInterface` dikonfigurasi menggunakan delegasi `by lazy`, yang berarti proses pembentukan objek Retrofit hanya akan dieksekusi sekali saja saat pertama kali dipanggil pada aplikasi (*lazy initialization*).

**Builder** digunakan untuk proses “merakit” Retrofit secara bertahap dengan rincian sebagai berikut: `.baseUrl(BASE_URL)` menetapkan alamat induk server (base url), `.addConverterFactory(GsonConverterFactory.create())` menyematkan mesin konverter Gson untuk mengubah format JSON menjadi *data class* secara otomatis, `.build()` memfinalisasi seluruh konfigurasi, dan `.create(ApiInterface::class.java)` mengeksekusi pembuatan implementasi pada `ApiInterface`. Dengan adanya struktur tersebut, bagian kode aplikasi lain (misal: `Repository` atau `ViewModel`) cukup memanggil `ApiClient.instance.getProducts()`.

## G. Membuat ViewModel

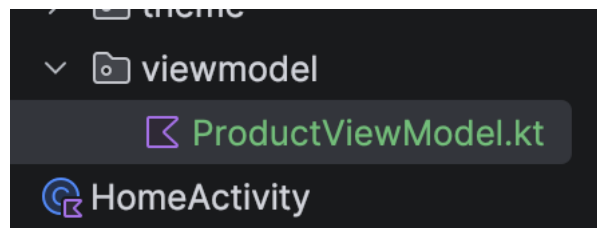
**ViewModel** adalah komponen arsitektur Android yang bertindak sebagai pengelola data dan logika bisnis yang berdiri di antara tampilan layar (*UI*) dan sumber data seperti API. Untuk membuat `ViewModel`, silakan ikuti tahapan-tahapan berikut:

### 1. Buat *package* baru di dalam *package* UI dengan nama *package* yaitu `viewmodel`



Gambar 15. Membuat Package `viewmodel`

## 2. buat Kotlin File bernama product view model di dalam package ViewModel



Gambar 16. Membuat Kotlin File Baru Bernama ProductViewModel

## 3. Pada File ProductViewModel.kt, implementasikan kode berikut:

```
sealed interface ProductUiState {  
    2 Usages  
    object Loading : ProductUiState  
    1 Usage  
    data class Success(val categories: List<Category>, val products: List<Product>) : ProductUiState  
    1 Usage  
    data class Error(val message: String) : ProductUiState  
}
```

Gambar 17. Membuat Sealed Class ProductUiInterface

Potongan kode di atas mengimplementasikan pola **UI State Pattern** menggunakan *sealed interface* di Kotlin untuk merepresentasikan seluruh kemungkinan kondisi status pada saat pengambilan data. Dalam arsitektur moder, antarmuka pengguna (UI) hanya bertugas mengamati dan melakukan render tampilan dengan satu kondisi status pada satu waktu. Dengan mendeklarasikan *ProductUiState* sebagai antarmuka tersegel (*sealed*), sistem membatasi turunan kelas hanya pada tiga skenario yang telah didefinisikan:

- **object Loading:** Kondisi saat aplikasi sedang melakukan *request* jaringan ke server yang akan menampilkan indikator *loading*.
- **data class Success(val categories: List<Category>, val products: List<Product>): ProductUiState:** Kondisi saat pemanggilan API berhasil. Data Class tersebut memberikan data berupa daftar kategori dan daftar produk, sehingga UI dapat langsung merender data kategori dan daftar kartu produk.
- **data class Error(val message: String): ProductUiState:** Kondisi saat terjadi kegagalan jaringan atau parsing (misalnya tidak ada koneksi internet atau *server error*). Kelas tersebut membawa informasi pesan *error* untuk ditampilkan ke pengguna.

4. Di bawah sealed class yang telah dideklarasikan di tahapan sebelumnya, implementasikan kode berikut:

```
class ProductViewModel : ViewModel() {
    private val _uiState = MutableStateFlow<ProductUiState>{ value = ProductUiState.Loading }

    val uiState: StateFlow<ProductUiState> = _uiState.asStateFlow()

    init {
        fetchData()
    }
    @Usage
    private fun fetchData() {
        viewModelScope.launch {
            _uiState.value = ProductUiState.Loading
            try {
                val categoriesResponse = ApiClient.instance.getCategories()
                val productsResponse = ApiClient.instance.getProducts()

                val mappedProducts = productsResponse.map { product ->
                    val matchedCategory = categoriesResponse.find { it.id == product.category_id }
                    product.copy(category = matchedCategory)
                }

                _uiState.value = ProductUiState.Success(
                    categories = categoriesResponse,
                    products = mappedProducts
                )
            } catch (e: Exception) {
                _uiState.value = ProductUiState.Error(
                    message = "Sagal memuat data: ${e.localizedMessage}"
                )
            }
        }
    }
}
```

Gambar 18. Deklarasi Class ViewModel

Potongan kode di atas mengimplementasikan kelas *ProductViewModel* yang bertindak sebagai jembatan logika bisnis antara lapisan data jaringan (*network*) dan antarmuka pengguna (*UI*) dengan memanfaatkan prinsip arsitektur MVVM dan Kotlin *Coroutines*. **Coroutines** merupakan library untuk melakukan proses di Kotlin yang memungkinkan eksekusi tugas-tugas berat di latar belakang tanpa membekukan antarmuka aplikasi (*UI Thread*).

Kelas *ProductViewModel* menerapkan pola enkapsulasi status (*state encapsulation*) melalui dua properti: *\_uiState* bertipe *MutableStateFlow* yang bersifat *private* agar hanya bisa dimodifikasi dari dalam *ViewModel*, serta *uiState* bertipe *StateFlow* publik (*read-only*) yang diekspos menggunakan *.asStateFlow()* agar lapisan UI dapat mengamati (*collect*) perubahan status tanpa dapat mengubah nilainya secara langsung. Status awal adalah *ProductUiState.Loading*, sehingga ketika UI memanggil *function* tersebut, maka tahapan pertama adalah menampilkan *loading*.

Ketika *ViewModel* pertama kali diinisialisasi, blok *init* secara otomatis mengeksekusi fungsi *fetchData()*. Di dalam blok *try*, aplikasi memanggil kedua fungsi API secara

berurutan: `getCategories()` dan `getProducts()`. Setelah kedua data berhasil diunduh, dilakukan operasi transformasi data menggunakan `.map` untuk memetakan setiap produk dengan kategori pasangannya. Jika seluruh proses berhasil, `_uiState.value` diperbarui menjadi `ProductUiState.Success` yang membawa daftar kategori dan produk yang sudah dipetakan. Sebaliknya, jika terjadi kendala seperti jaringan terputus atau server bermasalah, aliran eksekusi langsung ditangkap oleh blok `catch` (`e: Exception`) untuk memperbarui `_uiState.value` menjadi `ProductUiState.Error` lengkap dengan rincian pesan *error* yang terjadi.

## H. Modifikasi DetailProductScreen.kt

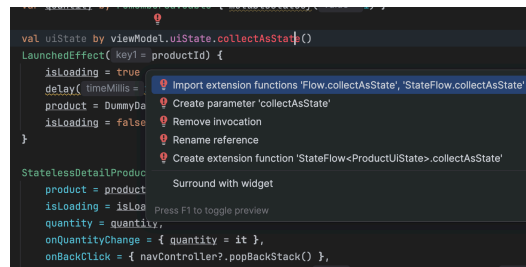
**Pada** tahapan ini, kita akan melakukan integrasi `DetailProductScreen` dengan `ViewModel` yang telah dibuat sebelumnya. Selain itu, kita juga akan menyesuaikan beberapa baris kode untuk mengintegrasikannya ke `ViewModel`.

### 1. Tambahkan Parameter *ProductViewModel* pada deklarasi function *DetailProductScreen*

```
@Composable
fun DetailProductScreen(productId: Int,
                        navController: NavController?,
                        viewModel: ProductViewModel) {
```

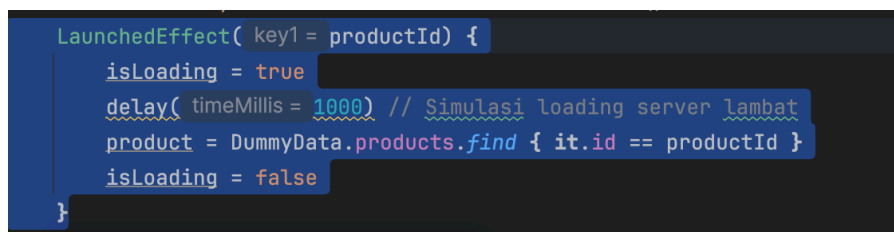
Gambar 19. Menambahkan parameter pada function `DetailProductScreen`

**2. Deklarasikan variabel UI State seperti pada gambar berikut dan lakukan import:**



Baris kode `val uiState by viewModel.uiState.collectAsState()` berfungsi untuk menghubungkan aliran data dari ViewModel ke fungsi Composable di Jetpack Compose secara reaktif. Fungsi ekstensi `collectAsState()` bertugas mendengarkan (*subscribe*) dan mengumpulkan nilai dari `StateFlow` (`viewModel.uiState`) secara terus-menerus, lalu mengonversinya menjadi objek State bawaan Compose. Penggunaan kata kunci delegasi `by` memungkinkan variabel `uiState` diakses langsung sebagai nilainya (yaitu instance dari `ProductUiState`), tanpa perlu memanggil `.value` secara manual. Melalui mekanisme tersebut, setiap kali ViewModel mengubah status status baru, mulai dari Loading, berpindah ke Success, atau Error. Jetpack Compose akan secara otomatis mendeteksi perubahan tersebut dan memicu proses penggambaran ulang layar (*recomposition*) hanya pada bagian antarmuka yang mengonsumsi data tersebut.

**3. Hapus kode simulasi loading yang telah diimplementasikan pada pertemuan sebelumnya**



Gambar 20. Menghapus kode simulasi loading

#### 4. Hapus pula parameter *isLoading* pada function *StatelessDetailProduct*

```
@Composable
fun StatelessDetailProduct(
    product: Product?, isLoading: Boolean, quantity: Int,
    onQuantityChange: (Int) -> Unit, onBackClick: () -> Unit, onAddToCartClick: () -> Unit
) {
    Scaffold(
```

Gambar 21. Menghapus parameter *isLoading* pada function *StatelessDetailProduct*

#### 5. Hapus blok *if & else if* yang terletak di function *StatelessDetailProduct*

```
        if (isLoading) {
            Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
                CircularProgressIndicator()
            }
        } else if (product != null) {
            Column(modifier = Modifier.fillMaxSize().padding(paddingValues).verticalScroll(

```

Gambar 22. Menghapus blok *if & else if* yang terletak di function *StatelessDetailProduct*

#### 6. Masih pada function *StatelessDetailProduct*, pada komponen *Column* yang sebelumnya di dalam scope *else if*, ganti dengan blok *if(product!=null)* seperti pada gambar berikut:

```
        ) { paddingValues ->
            if (product != null) {
                Column(modifier = Modifier.fillMaxSize().padding(paddingValues).verticalScroll(state = rememberScrollState())) {
                    val imageRes = if (product.img == "dummy_product") R.drawable.dummy_product else R.drawable.dummy_product
                    Image(painterResource(id = imageRes), contentDescription = null, modifier = Modifier.fillMaxWidth().height(280.dp))
                    Column(modifier = Modifier.padding(all = 16.dp)) {
                        Text(product.name, style = MaterialTheme.typography.headlineSmall, fontWeight = FontWeight.Bold)
                        Text("Rp ${product.price}", style = MaterialTheme.typography.titleLarge)
                        Spacer(modifier = Modifier.height(16.dp))
                        Text("Deskripsi", fontWeight = FontWeight.Bold)
                        Text(product.description ?: "")
                        Text("Stok: ${product.stock}")
                    }
                }
            }
        }
```

Gambar 23. Penyesuaian Blok *if*

7. Di dalam function `DetailProductScreen`, tambahkan blok kode berikut:

```
when (val state = uiState) {
    is ProductUiState.Loading -> {
        Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            CircularProgressIndicator()
        }
    }
    is ProductUiState.Error -> {
        Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            Text("Error: ${state.message}", color = MaterialTheme.colorScheme.error)
        }
    }
    is ProductUiState.Success -> {
        val product = state.products.find { it.id == productId }

        if (product == null) {
            Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
                Text("Produk tidak ditemukan.")
            }
        } else {
            StatelessDetailProduct(
                product = product,
                quantity = quantity,
                onQuantityChange = { newQuantity ->
                    quantity = newQuantity
                },
                onBackPressed = { navController?.popBackStack() },
                onAddToCartClick = {
                    Toast.makeText(context, "Membeli sebanyak $quantity", duration = Toast.LENGTH_SHORT).show()
                }
            )
        }
    }
}
```

Gambar 24. Implementasi Sealed Class untuk Pengkondisian Tampilan

Potongan kode pada gambar 24 memanfaatkan ekspresi percabangan `when (val state = uiState)` di Jetpack Compose untuk merender tampilan antarmuka (*UI rendering*) secara dinamis berdasarkan status terkini dari *Sealed Interface* `ProductUiState`. Inisiasi `val state = uiState` merupakan salah satu fitur pada Kotlin yaitu *smart casting* untuk mengenali tipe data spesifik di setiap blok tanpa perlu konversi manual.

Logika percabangan pada kode tersebut adalah sebagai berikut:

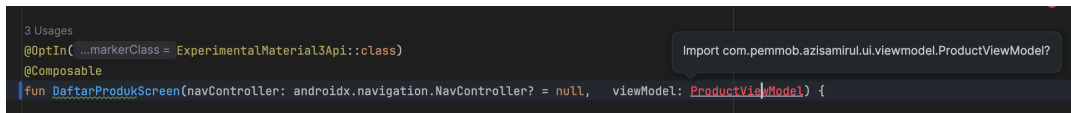
- **is ProductUiState.Loading:** Jika status masih memuat data, layar menampilkan indikator animasi berputar (`CircularProgressIndicator`) tepat di tengah layar penuh (`Box` dengan `contentAlignment = Alignment.Center`) sebagai tanda proses pengambilan data sedang berlangsung.
- **is ProductUiState.Error:** Jika terjadi kendala jaringan atau kegagalan pemanggilan API, layar menyajikan teks pesan kesalahan (`state.message`) dengan warna merah peringatan (`MaterialTheme.colorScheme.error`) agar pengguna mengetahui penyebab eror.

- **is ProductUiState.Success:** Ketika data berhasil didapatkan, kode mencari produk spesifik dari daftar state.products menggunakan `.find { it.id == productId }`. Jika produk tidak cocok/kosong, aplikasi akan menampilkan pesan *"Produk tidak ditemukan"*. Sebaliknya, jika data produk ditemukan, komponen composable `StatelessDetailProduct` akan dirender.

## I. Modifikasi Daftar Produk

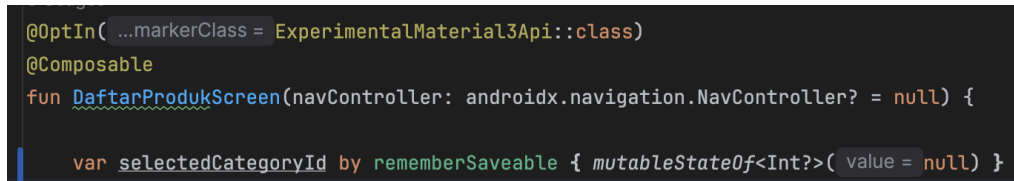
Pada bagian ini, kita akan melakukan modifikasi pada `DaftarProductScreen` sehingga terintegrasi dengan `ViewModel` seperti pada `DetailProductScreen`.

### 1. Tambahkan parameter `viewModel` pada deklarasi `DaftarProdukScreen`



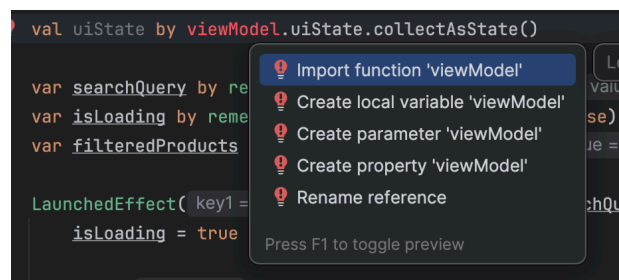
Gambar 25. Menambahkan Parameter `ViewModel` pada function `DaftarProdukScreen`

### 2. Tambahkan variabel `selectedCategoryId`



Gambar 26. Tambahkan Variabel `SelectedCategoryId`

### 3. Tambahkan variabel `uiState`



Gambar 27. Deklarasi Variabel `uiState`

#### 4. Hapus Blok *LaunchedEffect*

```
LaunchedEffect( key1 = selectedCategoryId, key2 = searchQuery) {
    isLoading = true

    delay( timeMillis = 1000)

    val filteredByCategory = if (selectedCategoryId != null) {
        state.products.filter { it.category_id == selectedCategoryId }
    } else {
        state.products
    }

    filteredProducts = if (searchQuery.isBlank()) {
        filteredByCategory
    } else {
        filteredByCategory.filter { it.name.contains( other = searchQuery, ignoreCase = true) }
    }

    isLoading = false
}
```

Gambar 28. Penghapusan Blok *LaunchedEffect*

#### 5. Di dalam function *DaftarProdukScreen*, tambahkan kode *when* untuk mengetahui kondisi *uiState*.

```
@Composable
fun DaftarProdukScreen(navController: androidx.navigation.NavController? = null, viewModel: ProductViewModel) {

    var selectedCategoryId by rememberSaveable { mutableStateOf<Int?>( value = null) }
    val uiState by viewModel.uiState.collectAsState()

    var searchQuery by rememberSaveable { mutableStateOf( value = "" ) }
    var isLoading by remember { mutableStateOf( value = false) }
    var filteredProducts by remember { mutableStateOf( value = emptyList<Product>()) }

    when(val state=uiState){
        is ProductUiState.Loading->{

        }
        is ProductUiState.Error->{

        }

        is ProductUiState -> {

        }
    }
}
```

Gambar 29. Penambahan kode *When*

#### 6. Isikan Blok *When* seperti pada gambar berikut:

```
when (val state = uiState) {
    is ProductUiState.Loading -> {
        Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            CircularProgressIndicator()
        }
    }

    is ProductUiState.Error -> {
        Box(modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            Text("Error: ${state.message}", color = MaterialTheme.colorScheme.error)
        }
    }
}
```

Gambar 30. Pengisian blok *when* pada status *UiState Loading & Error*

**7. Lanjutkan pengisian Blok When untuk uiState success:**

```
is ProductUiState.Success -> {
    if (selectedCategoryId == null && state.categories.isNotEmpty()) {
        selectedCategoryId = state.categories.first().id
    }

    val filteredByCategory = if (selectedCategoryId != null) {
        state.products.filter { it.category_id == selectedCategoryId }
    } else {
        state.products
    }

    val filteredProducts = if (searchQuery.isBlank()) {
        filteredByCategory
    } else {
        filteredByCategory.filter {
            it.name.contains(other = searchQuery, ignoreCase = true)
        }
    }
}
```

*Gambar 31. pengisian Blok When untuk uiState Success*

**8. Masih di dalam blok success, sesuaikan kode dari pertemuan sebelumnya menjadi seperti gambar di bawah ini:**

```
val filteredProducts = if (searchQuery.isBlank()) {
    filteredByCategory
} else {
    filteredByCategory.filter {
        it.name.contains(other = searchQuery, ignoreCase = true)
    }
}

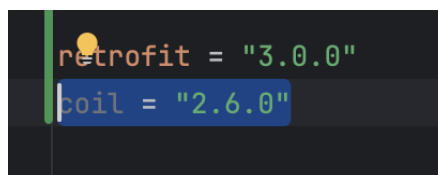
StatelessDaftarProduct(
    categories = state.categories,
    selectedCategoryId = selectedCategoryId,
    onCategorySelected = { selectedCategoryId = it },
    searchQuery = searchQuery,
    onSearchQueryChange = { searchQuery = it },
    isLoading = false,
    products = filteredProducts,
    onProductClick = { product ->
        navController?.navigate( route = "detail/${product.id}")
    },
    onContactUsClick = {
        navController?.navigate( route = "hubungi_kami")
    }
)
```

*Gambar 32. Implementasi lanjutan blok uiState Success*

## J. Memuat gambar yang bersumber dari internet

Pada bagian ini, kita akan mengimplementasikan proses untuk memuat gambar dari internet menggunakan library Coil. Coil (*Coroutine Image Loader*) adalah library pemuatan gambar yang memanfaatkan Kotlin Coroutines untuk mengunduh dan menampilkan gambar dari internet secara asinkron tanpa membekukan antarmuka aplikasi (*UI thread*). Melalui integrasi alaminya di Jetpack Compose lewat komponen seperti *AsyncImage*, pengembang cukup menyematkan URL gambar dari peladen (seperti tautan gambar produk dari respons API) ke parameter model.

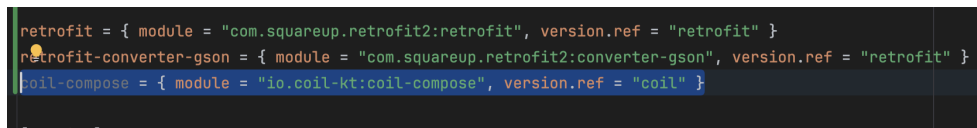
### 1. Tambahkan script berikut di file `libs.version.toml` pada segmen `[versions]`



```
retrofit = "3.0.0"
coil = "2.6.0"
```

Gambar 33. Deklarasi Versi Coil

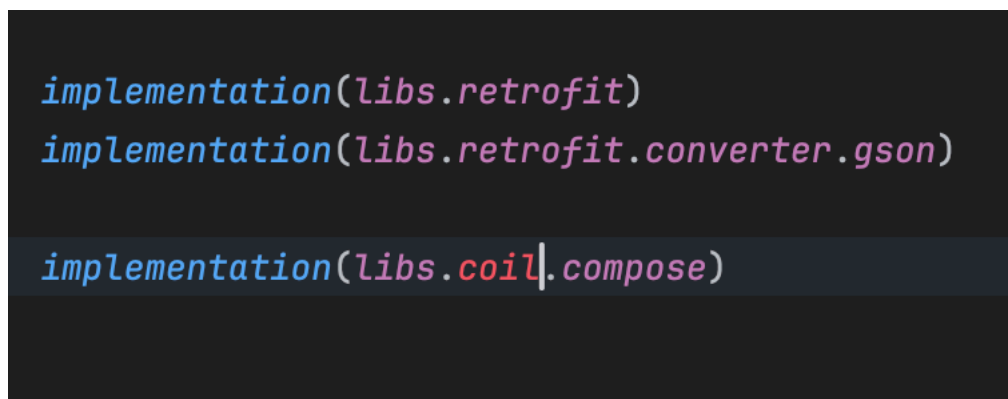
### 2. Tambahkan script berikut di file `libs.version.toml` pada segmen `[libraries]`



```
retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
retrofit-converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "retrofit" }
coil-compose = { module = "io.coil-kt:coil-compose", version.ref = "coil" }
```

Gambar 34. Menambahkan library coil

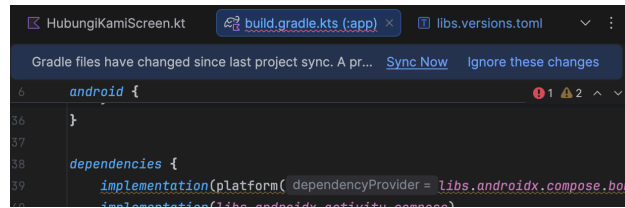
### 3. Tambahkan script berikut di file `build.gradle.kts` pada segmen `dependencies` seperti pada saat mengimplementasikan library retrofit.



```
implementation(libs.retrofit)
implementation(libs.retrofit.converter.gson)
implementation(libs.coil.compose)
```

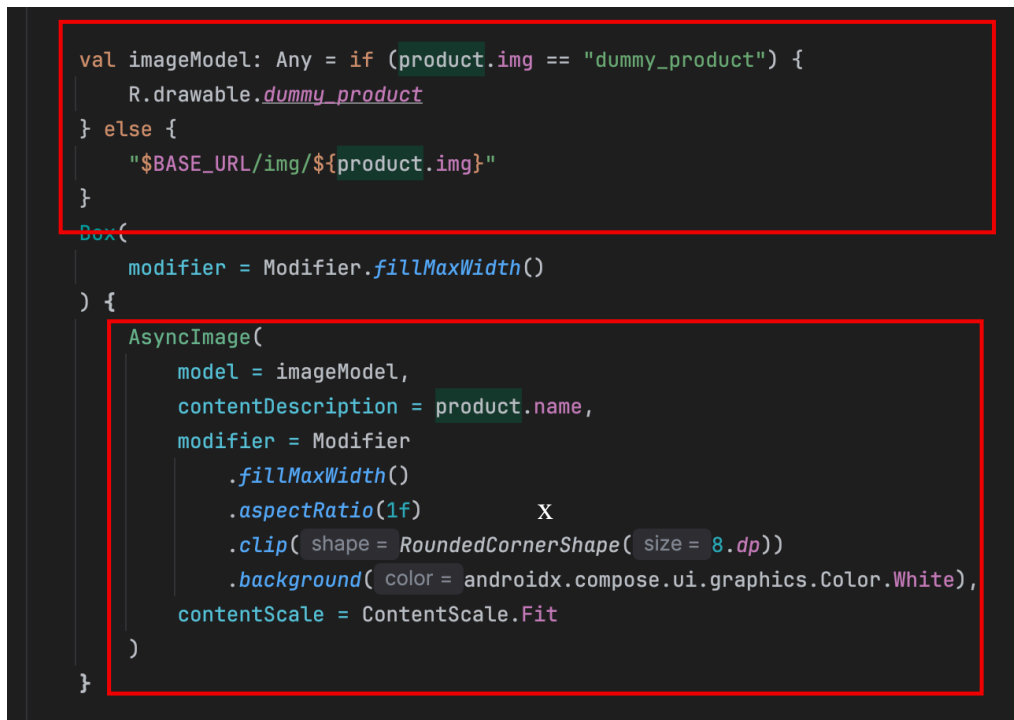
Gambar 35. Implementasi Library Coil pada `build.gradle.kts`

4. Lakukan sync project with gradle dan tunggu beberapa saat



Gambar 36. Proses Sync Project dengan Gradle

5. Ganti blok image pada StatelessDetailProduk menjadi seperti gambar berikut:



Gambar 37. Perubahan Komponen Image StatelessDetailProduk

6. Ganti pula komponen image pada bagian `ProductItemCard` pada file `DetailProductScreen` dengan kode berikut:

```
@Composable
fun ProductItemCard(product: Product, onClick: () -> Unit) {
    Card(
        modifier = Modifier
            .padding(all = 8.dp)
            .fillMaxWidth()
            .clickable { onClick() },
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)
    ) {
        Column(modifier = Modifier.padding(all = 12.dp)) {
            val imageModel: Any = if (product.img == "dummy_product") {
                R.drawable.dummy_product
            } else {
                BASE_URL + "img/${product.img}"
            }

            Box(
                modifier = Modifier.fillMaxWidth()
            ) {
                coil.compose.AsyncImage(
                    model = imageModel,
                    contentDescription = product.name,
                    modifier = Modifier
                        .fillMaxWidth()
                        .aspectRatio(1f)
                        .clip(shape = RoundedCornerShape(size = 8.dp))
                        .background(color = androidx.compose.ui.graphics.Color.White),
                    contentScale = ContentScale.Fit
                )
            }
        }
    }
}
```

Gambar 38. Perubahan Komponen Image pada `ProductItemCard`

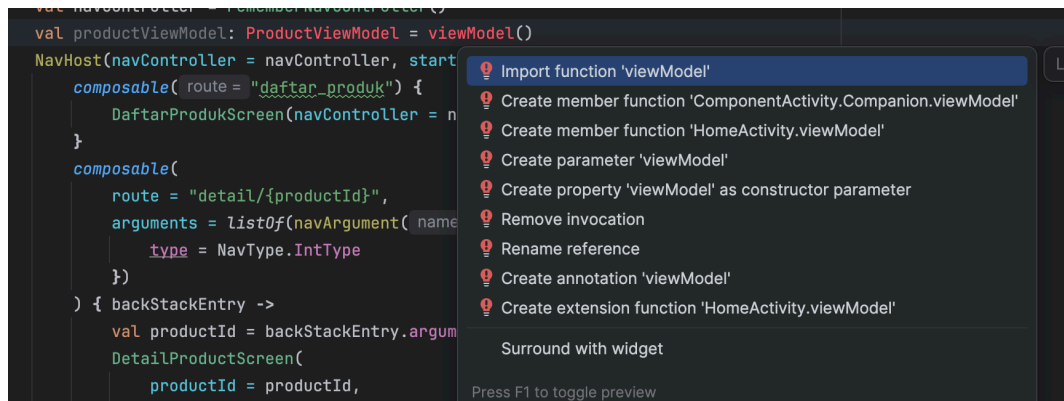
7. Jika kode mengalami *error* karena belum dilakukan *import*, maka lakukan *import* dari *class* berikut:

```
val imageModel: Any = if (product.img == "dummy_product") {
    R.drawable.dummy_product
} else {
    "$BASE_URL/img/${product.img}"
}

Box(
    // Import property 'JualanConstants.BASE_URL'
```

Pada bagian ini, kita akan menyesuaikan proses navigasi antar halaman dengan penyesuaian parameter.

1. Buka `HomeActivity.kt` dan tambahkan variabel seperti pada gambar berikut serta lakukan import `ProductViewModel` serta `viewModel`



*Gambar 39. Deklarasi Variabel ProductViewModel*

2. Pada pemanggilan *function* `DetailProductScreen` dan `DaftarProductScreen` yang terdapat di `HomeActivity.kt`, tambahkan parameter *viewModel* dengan nilai `productViewModel` sesuai yang telah dideklarasikan pada tahapan sebelumnya.



Gambar 40. Modifikasi Kode pada HomeActivity

**JALANKAN APLIKASI TERSEBUT DI *EMULATOR*/PERANGKAT ANDA**